

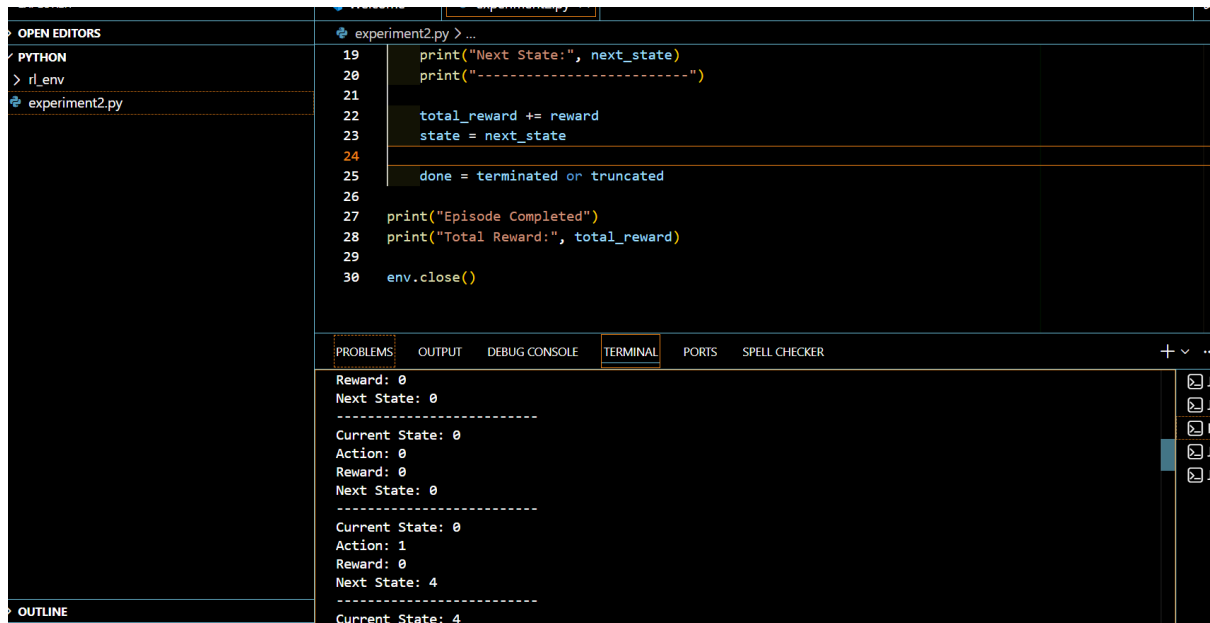
M.GUNA SHEKAR

192472345

MLA0301

REINFORCEMENT LEARNING

Experiment 2: Installation of Reinforcement Learning Development Environment:-

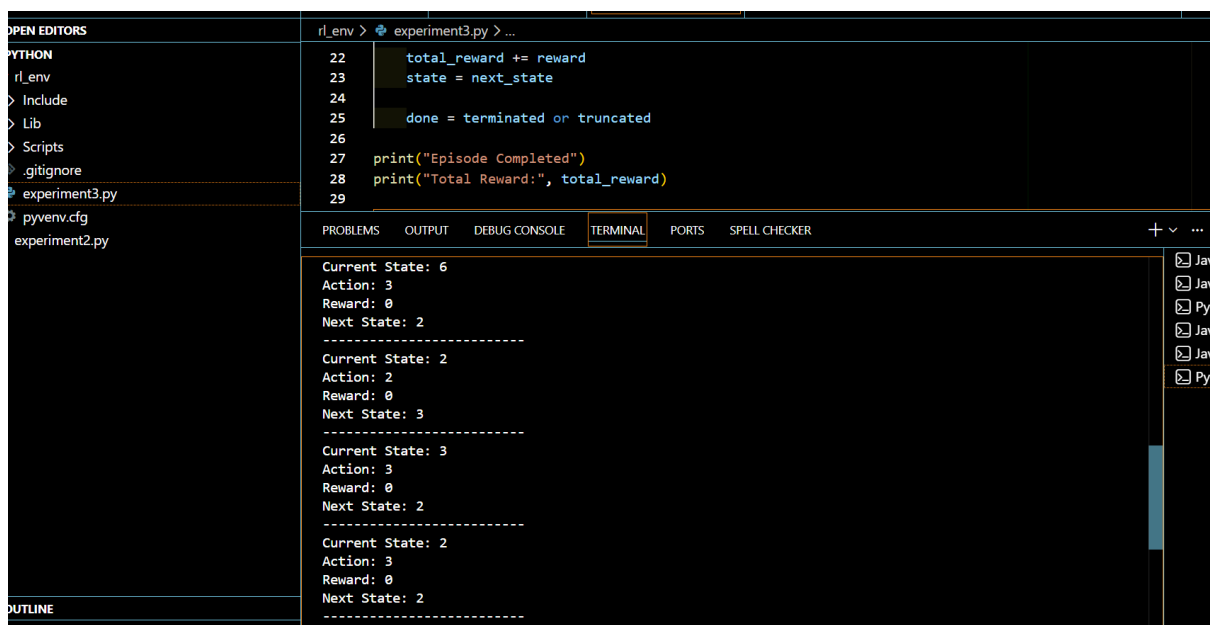


```
19 print("Next State:", next_state)
20 print("-----")
21
22 total_reward += reward
23 state = next_state
24
25 done = terminated or truncated
26
27 print("Episode Completed")
28 print("Total Reward:", total_reward)
29
30 env.close()
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER

```
Reward: 0
Next State: 0
-----
Current State: 0
Action: 0
Reward: 0
Next State: 0
-----
Current State: 0
Action: 1
Reward: 0
Next State: 4
-----
Current State: 4
```

Experiment 3: Implementation of the Reinforcement Learning Framework:-



```
22 total_reward += reward
23 state = next_state
24
25 done = terminated or truncated
26
27 print("Episode Completed")
28 print("Total Reward:", total_reward)
29
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER

```
Current State: 6
Action: 3
Reward: 0
Next State: 2
-----
Current State: 2
Action: 2
Reward: 0
Next State: 3
-----
Current State: 3
Action: 3
Reward: 0
Next State: 2
-----
Current State: 2
Action: 3
Reward: 0
Next State: 2
-----
```

Experiment 4: Simulation of a Markov Decision Process (MDP):-

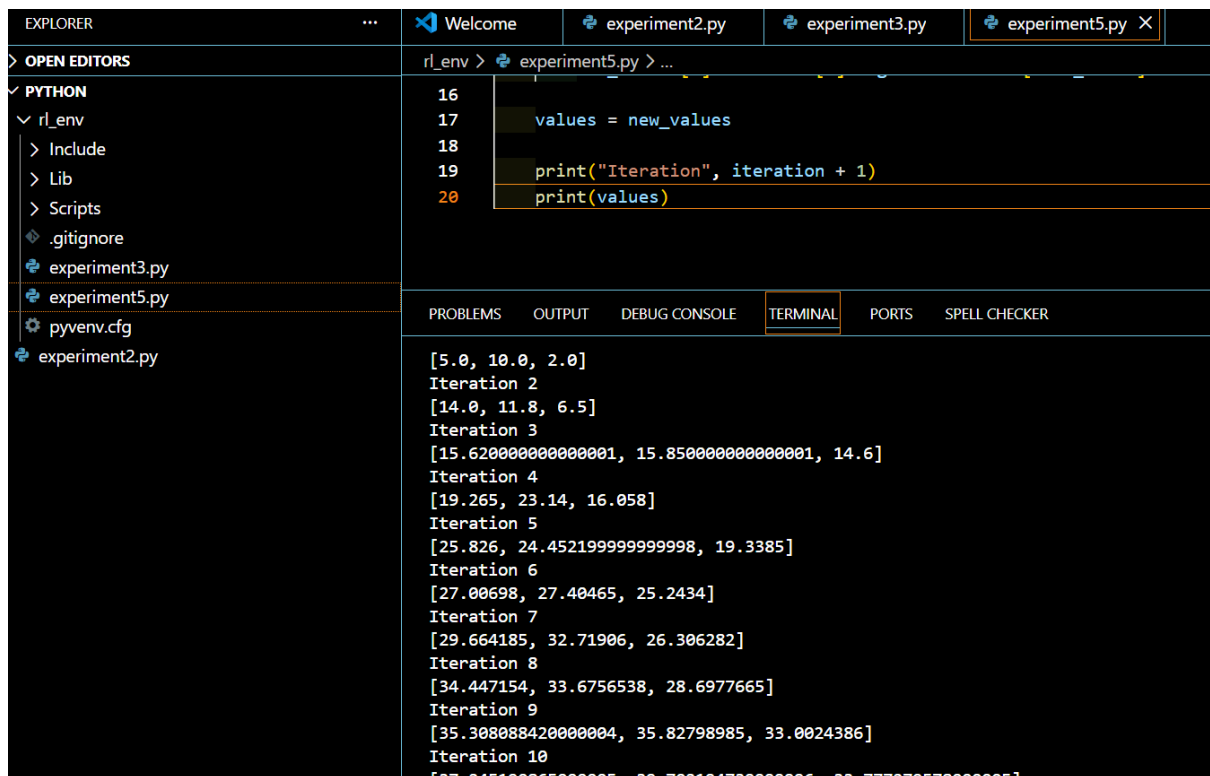
The screenshot shows a VS Code editor with the following components:

- EXPLORER:** A file tree on the left showing a project structure with files like `rl_env`, `Include`, `Lib`, `Scripts`, `.gitignore`, `experiment3.py`, `experiment4.py`, `pyvenv.cfg`, and `experiment2.py`.
- EDITOR:** The main window displays the code for `experiment4.py`. The code is as follows:

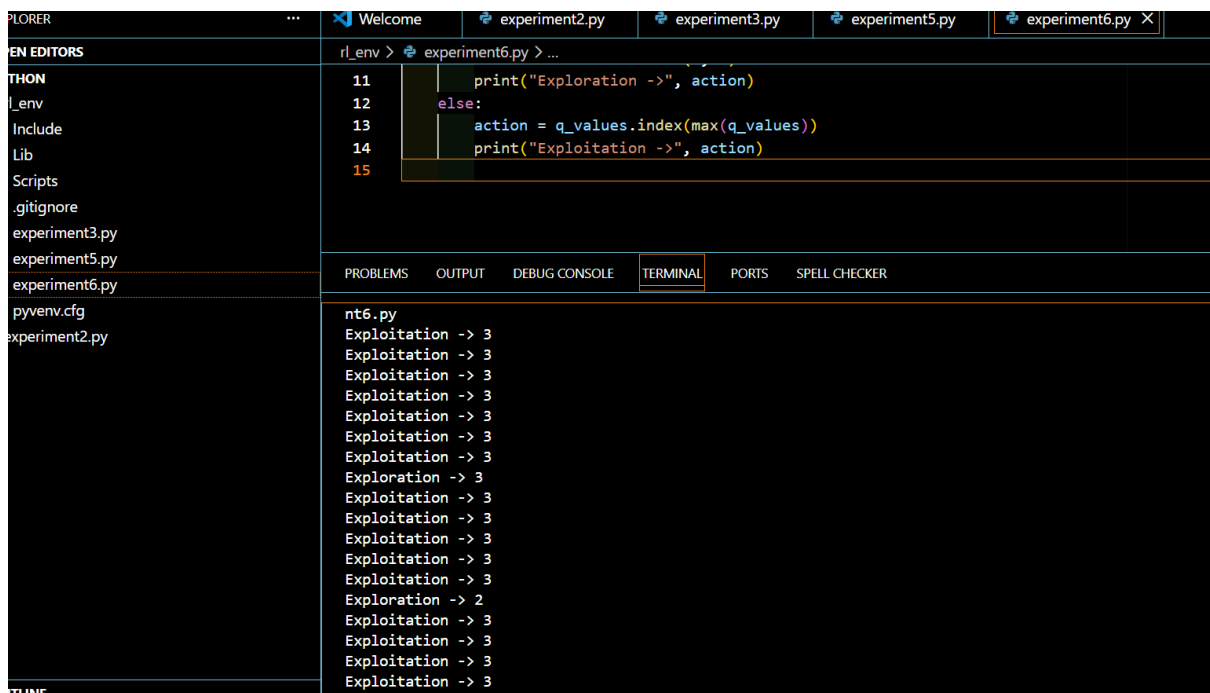
```
16     print("Next State:", next_state)
17     print("Reward:", reward)
18     print("-----")
19
20     state = next_state
```
- TERMINAL:** The bottom panel shows the output of the program. It displays a sequence of states and rewards:

```
Reward: 10
-----
Current State: C
Next State: A
Reward: 2
-----
Current State: A
Next State: B
Reward: 5
-----
Current State: B
Next State: C
Reward: 10
-----
Current State: C
Next State: A
Reward: 2
-----
Current State: A
Next State: B
Reward: 5
```

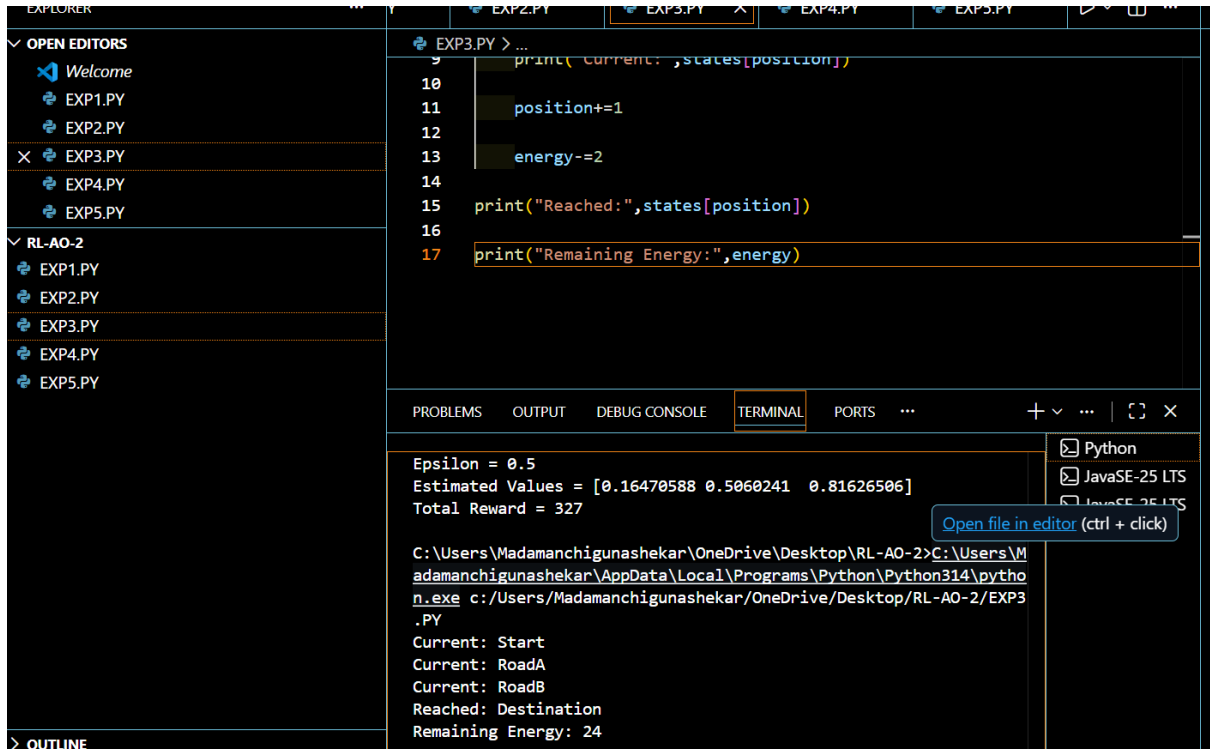
Experiment 5: Bellman Equation Demonstration:-



Experiment 6: Exploration vs. Exploitation Strategies:-



3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.



```
EXP3.PY > ...
9      print("Current: ",states[position])
10
11      position+=1
12
13      energy-=2
14
15      print("Reached:",states[position])
16
17      print("Remaining Energy:",energy)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ...

Epsilon = 0.5
Estimated Values = [0.16470588 0.5060241 0.81626506]
Total Reward = 327

C:\Users\Madamanchigunashekar\OneDrive\Desktop\RL-AO-2>C:\Users\Madamanchigunashekar\AppData\Local\Programs\Python\Python314\python.exe c:/Users/Madamanchigunashekar/OneDrive/Desktop/RL-AO-2/EXP3.PY

Current: Start
Current: RoadA
Current: RoadB
Reached: Destination
Remaining Energy: 24

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

The screenshot shows the Visual Studio Code interface with the Explorer, Open Editors, and Terminal panels. The Explorer panel shows a project named 'RL-AO-2' with files EXP1.PY through EXP5.PY. The Open Editors panel shows EXP4.PY selected. The code in EXP4.PY is as follows:

```

7  value=np.zeros(4)
8
9  for iteration in range(20):
10
11     new=value.copy()
12
13     for s in range(3):
14
15         new[s]=reward[s]+gamma*value[s+1]
16
17     value=new
18
19     print("Optimal State Values")
20

```

The Terminal panel shows the output of running EXP4.PY:

```

.PY
Current: Start
Current: RoadA
Current: RoadB
Reached: Destination
Remaining Energy: 24

C:\Users\Madamanchigunashekar\OneDrive\Desktop\RL-AO-2>C:\Users\Madamanchigunashekar\AppData\Local\Programs\Python\Python314\python.exe c:/Users/Madamanchigunashekar/OneDrive/Desktop/RL-AO-2/EXP4.PY
Optimal State Values
[-1.71 -1.9 -1.  0. ]

```

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

The screenshot shows the Visual Studio Code interface with the Explorer, Open Editors, and Terminal panels. The Explorer panel shows a project named 'RL-AO-2' with files EXP1.PY through EXP5.PY. The Open Editors panel shows EXP5.PY selected. The code in EXP5.PY is as follows:

```

15  reward+=10
16
17  reward+=battery//5*2
18
19  print("Deliveries Completed =",deliveries)
20
21  print("Remaining Battery =",battery)
22
23  print("Reward =",reward)

```

The Terminal panel shows the output of running EXP5.PY:

```

adamanchigunashekar\AppData\Local\Programs\Python\Python314\python.exe c:/Users/Madamanchigunashekar/OneDrive/Desktop/RL-AO-2/EXP4.PY
Optimal State Values
[-1.71 -1.9 -1.  0. ]

C:\Users\Madamanchigunashekar\OneDrive\Desktop\RL-AO-2>C:\Users\Madamanchigunashekar\AppData\Local\Programs\Python\Python314\python.exe c:/Users/Madamanchigunashekar/OneDrive/Desktop/RL-AO-2/EXP5.PY
Deliveries Completed = 6
Remaining Battery = 0
Reward = 60

```

